Lab 51: Refund amount jumped to the wrong order

Topic 5: Context Management & Reliability · Task 5.1: Manage conversation context to preserve critical information across long interactions

Runs on Simulator 1: Customer Support Resolution Agent · Run in class

Source: SkillCertPro practice exam Part 1, Q40 (unofficial third-party question)

The question

Production transcripts from the support resolution agent show that, after long conversations involving multiple orders, the agent sometimes applies a refund amount from one issue to a different order after older turns have been summarized. What change would most effectively preserve the critical issue details across the session?

A. Summarize the complete conversation more frequently and replace prior summaries whenever the context grows large.

B. Maintain a structured case-facts block that is updated after each relevant turn and made available on every relevant turn.

C. Add a system instruction telling the agent to remember all order IDs and refund amounts throughout the case.

D. Place the most recent order-lookup result at the end of the prompt before the agent response.

Simulate it live

```
# In the browser: open Simulator 1, then Lab 51
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 51

# In a terminal, from simulators/simulator-1-customer-support/
uv run support_agent.py --case CS-12
```

What you should see

After three turns the only refund is $45 on ORD-5503.

evidence/runs/CS-12.json: three turn_replies each opening with CASE FACTS; the only refund is $45 on ORD-5503.

Open `simulators/simulator-1-customer-support/evidence/` and find it yourself.

The answer

B. A case-facts block updated and shown on every turn.

Separate critical case state from the conversational transcript. A structured facts block (customer_id, order_id, disputed amount, refund status, customer statements, policy decisions, open issues) is updated by the orchestration layer as evidence arrives and supplied whenever decisions depend on exact facts. The Agent SDK adds session persistence and automatic compaction, but compaction and summarisation are lossy for precise transactional details. Use summaries for low-value dialogue and keep exact data as structured state your application owns.

Why the others fail

A. More frequent summarisation means more lossy compression of exactly the details that are going wrong.

C. An instruction to “remember” adds no durable context layer.

D. A recency trick that covers only the latest lookup. Facts about the other orders are still lost.

Check yourself

- every turn's reply repeats the three order ids and amounts
- the lamp refund is deferred, the duplicate charge is refunded

Now change one thing

Copy a file in cases/ to a new CS id, change the customer's message and expected disposition, rerun with --case, then score it with evaluate.py.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.